Command	Definition
git status	This command shows the current status of the working directory and staging area. The working directory is where you make changes to your project, and the staging area (or index) is where you place changes you want to commit. It lets you see which changes have been staged (added to the index), which haven't (still in the working directory), and which files aren't being tracked by Git (new files).
git add .	This command adds all the changes in the current directory (and subdirectories) to the staging area. The staging area is like a prep area where you can group changes before committing them to the repository.
git commit -m "message"	This command records the changes in the staging area to the repository with a descriptive message provided in quotes. A repository is a storage location for your project's code and history of changes. The message explains what changes were made.
git push origin branch_name	This command uploads the local repository changes to the remote repository named "origin" on the specified branch. A remote repository is a version of your project that is hosted on the internet or another network. A branch is a separate line of development within your project.
git pull origin branch_name	This command fetches and integrates changes from the remote repository named "origin" to the specified branch in the local repository. It's like a combination of git fetch (which downloads changes) and git merge (which integrates changes).
git diff	This command shows the differences between the files in the working directory and the staging area. It helps you see what changes you have made that are not yet staged or committed.
git log	This command shows the commit history for the current branch. Each commit is a snapshot of your repository at a specific point in time, and the history includes details like the commit message, author, and date.
git branch	This command lists all the branches in your repository and shows which branch you are currently on. A branch is a separate line of development within your project.
git checkout branch_name	This command switches the current branch to the specified branch. It changes the working directory to match the specified branch.
git merge branch_name	This command merges changes from the specified branch into the current branch. It integrates the histories of the two branches.
git fetch origin	This command downloads changes from the remote repository named "origin" but does not integrate them into the local repository. It's like updating your knowledge of the remote repository without making any changes locally.

git remote -v	This command shows the URLs of the remote repositories. A remote repository is a version of your project that is hosted on the internet or another network.
git reset --hard commit_hash	This command resets the working directory, the staging area, and the current branch to the specified commit. The commit hash is a unique identifier for a commit. This is a powerful command that can undo changes.
git stash	This command temporarily saves changes in the working directory that are not ready to be committed. It helps you clean your working directory without committing your changes.
git stash pop	This command restores the most recently stashed changes to the working directory and removes the changes from the stash. The stash is like a temporary storage area for changes you want to save for later.
git clone repository_url	This command creates a copy of an existing repository in a new directory. The repository URL is the address of the remote repository you want to clone.
git config --global user.email "you@example.com"	This command sets the global email address that will be associated with your Git commits. The global option means this setting will apply to all repositories on your machine.
git config --global user.name "Your Name"	This command sets the global username that will be associated with your Git commits. The global option means this setting will apply to all repositories on your machine.
git init	Initializes a new Git repository in the current directory. This is often the first command when starting to use Git in a project.
git rm --cached filename	Removes a file from the staging area without deleting it from the working directory. This is useful if you accidentally add a file you don't want to commit.
git rebase branch_name	Re-applies commits on top of another base branch. It's used to keep a linear commit history by applying changes from one branch to another.
git revert commit_hash	Creates a new commit that undoes the changes from the specified commit. Unlike git reset, it preserves the commit history.
git tag tag_name	Creates a tag, which is a reference to a specific commit. Tags are often used to mark release points (e.g., v1.0).
git cherry-pick commit_hash	Applies the changes from a specific commit onto the current branch. Useful for applying individual changes without merging an entire branch.

git log --oneline	Displays the commit history in a condensed format, showing just the commit hash and message in a single line each. Helpful for a quick overview.
git log --oneline filepath	Displays the commit history in a condensed format, showing just the commit hash and message in a single line each for that file whose path is provided. Helpful for a quick overview
git show <commit hash>:path/to/your/file.py	Displays how that file was like on a specific commit hash which we found from the above command
git checkout <commit hash>-- path/to/your/file.py	Revert the file to that commit hash
git log --before="<date>" -- path/to/your/file.py	Shows the commit before certain date
git reset filename	Removes a file from the staging area while keeping the changes in the working directory. Useful to unstage files without discarding changes.
git log prod-erp --since="2025-12-23" --until="2026-01-01 23:59" --pretty=format:"%h,%an,%ad,%s" --date=short >> prod-erp-commit-report.csv	This command pulls all the commits within a specific date range from the commit history of the given branch and converts them to a CSV-friendly format and saves or appends them to a file; this output generally contains the commit short hash, author name, commit date and commit message, so that you can easily analyse the data in Excel or any spreadsheet tool for release reports, deployment summary, audit or team-wise contribution tracking.